

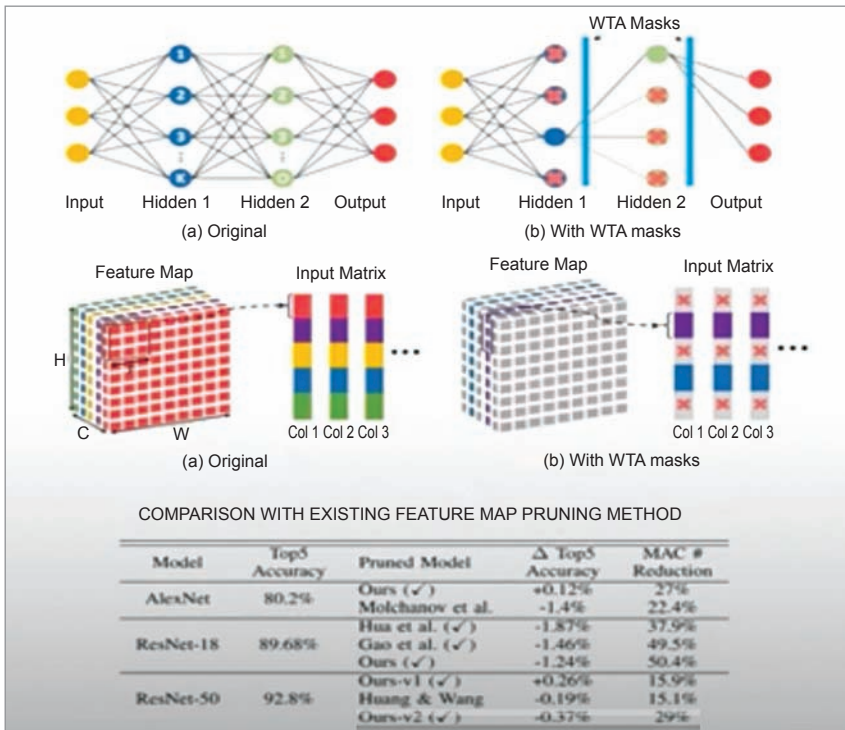


Figure 5: Comparison with existing feature map pruning method

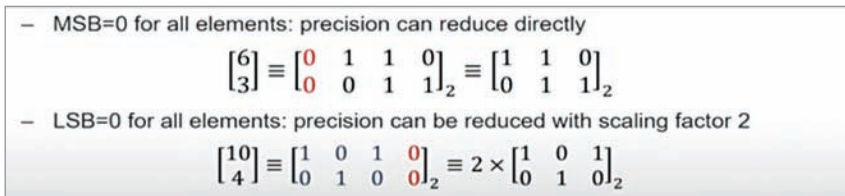


Figure 6: Mixing precision with bit-level sparsity

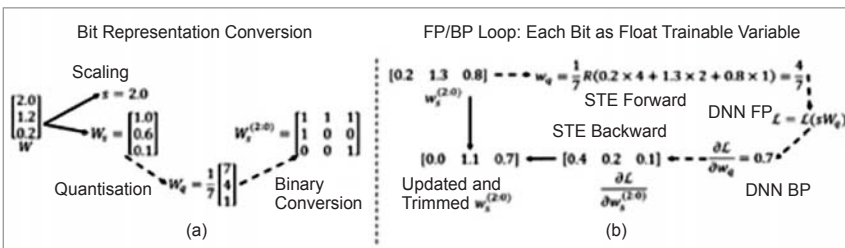


Figure 7: Bit representation conversion

Let us now look at pruning and sparsification for deep neural networks (DNN) acceleration. Why do we need to sparse DNN models? Some answers are:

- Because the ever-growing size brought challenges to the deployment of the DNN models.
- A lot of studies have shown that the parameters of DNNs are redundant. Many of them can be pruned and

reduced so that we can reduce the computation and bandwidth requirement by eliminating that redundancy. This raises the question as to what solution do we have in order to achieve the goal.

When we talk about sparsification and DNNs, there are multiple ways to achieve the goal. We can sparse weight parameters through unstructured

prunings or structured prunings. We can also consider activation sparsity where ReLU has been widely adopted.

Our lab has proposed bit-level sparsity, which basically decomposes the weight parameter into binary presentations that aims to remove unwanted zeros from the computation. Then there is inherent sparsity that transposes the convolutions and expands the flick necklace convolutional kernels. Hence, we do not have to involve those zeroes into the computation.

There is a need for structural sparsity because non-structured sparsity may not speedup traditional platforms like GPUs. If we are able to achieve structured sparse state, then we are printing out the weight parameters by rows or columns that will be more friendly to the hardware. These structures can be squeezed into much smaller matrices for the operations. Let us see how we can do that.

Structurally sparse deep neural network

The key idea here is to form a group. A group can be any form of a weight block, depending on what sparse structure you want to learn. For example, in CNNs, a group of weights can be a channel, a 3D filter, a 2D filter, a filter shape fibre (a weight column), and even a layer (in ResNets), as shown in Figure 3.

Once the groups have been defined, we can apply lasso regulation to the groups. For example, Figure 4 shows a weight vector with five elements, which we divide into two groups—one with two and the other with three elements.

We can apply group lasso on each group by following the equations illustrated in the figure itself. Then we sum the group lasso over all groups as a regulation and add it as a part of the optimisation terms of the overall optimisation objectives. This forms the procedure of our method called the Structure Sparsity Learning.